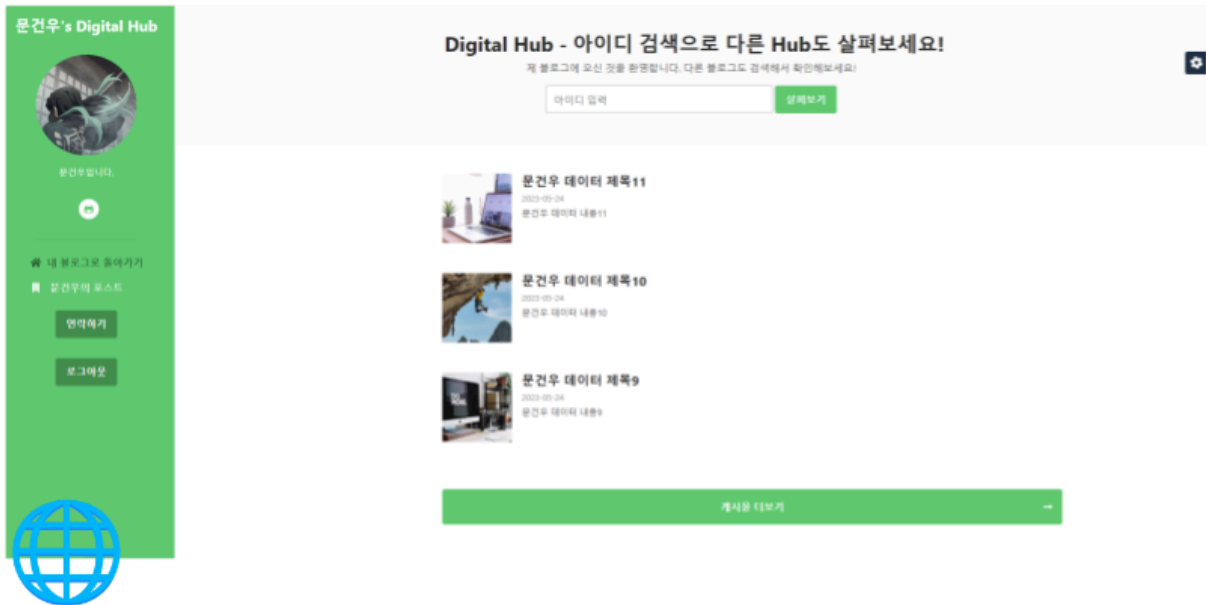




BootBlog

📅 생성일	2023년 5월 9일 → 2023년 5월 23일
☰ 태그	HTML JavaScript SpringBoot Thymeleaf
≡ 한 줄 설명	Spring Boot를 활용해 회원가입부터 글 작성·댓글·프로필 탐색까지 하드코딩으로 구현한 멀티 유저 블로그 플랫폼
≡ 개발 인원(역할)	4명 (프론트·백 구분 없이 풀스택 / 백엔드 주력 2명)
≡ 관련 활동	풀스택 개발자 6개월 국비지원 교육과정 팀 프로젝트
🔗 발표 자료	SpringProject_.pptx

🔗 Link / Source

GitHub — <https://github.com/rilac/bootblog>

📖 상세 내용

- 사용대상
블로그 서비스를 이용하는 일반 사용자. 회원가입 후 자신만의 블로그를 운영하고, 다른 사용자의 블로그를 방문해 글을 읽고 댓글을 남길 수 있다.

- 학습 배경

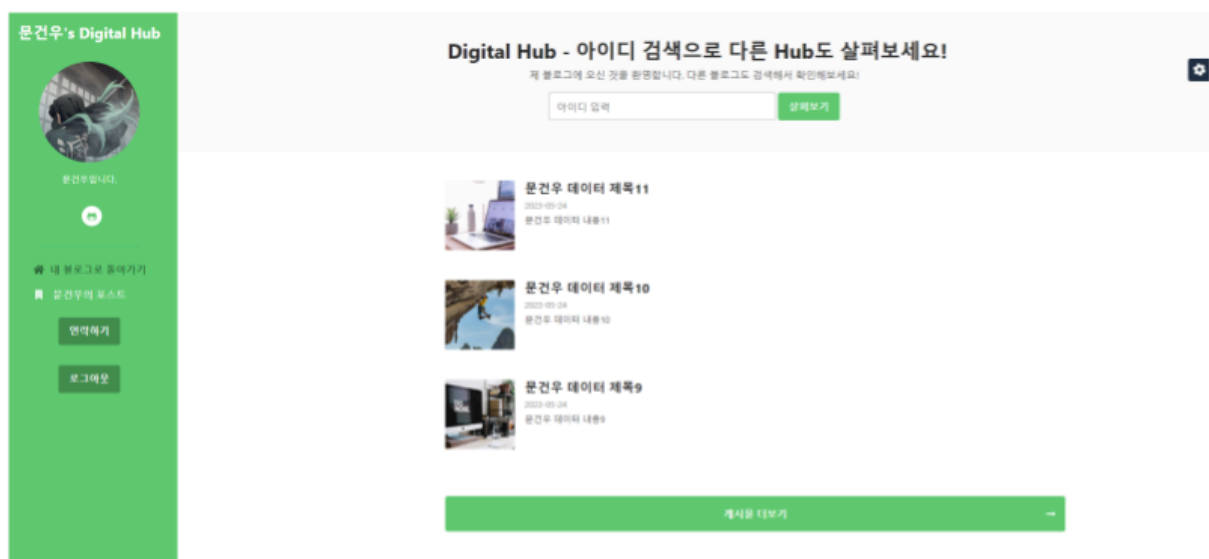
Spring Framework를 처음 학습하는 단계에서, 이론을 예제 수준에서 끝내지 않고 "사용자가 실제로 써본 적 있는 서비스 유형"을 직접 구현해 보자는 것이 프로젝트의 출발점. 티스토리·벨로그 같은 기존 블로그 플랫폼을 레퍼런스로 두고, 핵심 기능들을 최대한 비슷하게 재현하는 것을 목표로 했다.

- 구현 접근

Spring MVC 기본 요청 흐름 위에 JPA/Hibernate로 도메인 모델을 설계하고, `HttpSession` 기반 인증 + `HandlerInterceptor` 로 보호 경로를 제어하는 표준적인 구조를 따랐다. 화면은 Thymeleaf + Bootstrap으로 구성해, 다중 테마(theme-1 ~ theme-8) 전환까지 구현해 완성도를 높였다.

- 학습 성과

회원가입·로그인, 블로그 글 CRUD, 댓글, 페이지네이션, 다른 사용자 블로그 탐색 등 블로그 플랫폼이 공통적으로 가진 기능을 한 번의 프로젝트 사이클로 경험함으로써, 이후 더 복잡한 Spring Boot 프로젝트(SSAFY 익명 커뮤니티 플랫폼 등)로 확장해 나갈 수 있는 기반을 마련했다.



사용 기술 및 라이브러리

- **Backend:** Java 11, Spring Boot 2.7.11, Spring MVC, Spring Data JPA, Hibernate
- **DB:** Oracle Database (ojdbc8)
- **View:** Thymeleaf, Bootstrap, jQuery

- 기타: Lombok, Spring Validation, Gradle
-

역할

- ERD 설계 및 JPA 기반 테이블 생성 :

`Members`, `Board`, `Comments` 엔티티 관계 설계 후 `ddl-auto: create` 설정으로 스키마 자동 생성

- DTO 계층 설계 :

`LoginDTO`, `RegisterDTO`, `BoardDTO`, `CommentDTO`, `UpdateDTO` 등 요청/응답 분리 구조로 Entity 직접 노출 방지

- 댓글 기능 추가 구현 :

`Comments` 엔티티와 `Board` 간 `@ManyToOne` 매핑, 작성/삭제 API와 게시물 상세 페이지 연동

- `LoginCheckInterceptor` 구현 :

`HandlerInterceptor` 를 구현해 `WebConfig` 에 등록, 비로그인 사용자의 보호 경로 접근 차단 및 리다이렉트 처리

- 테스트 데이터 초기화 :

`TestDataInit` 을 통해 애플리케이션 구동 시 샘플 회원·글·댓글을 자동 주입해 반복 개발·검증 효율화

- PM 역할 :

일정 관리, 역할 분배, 코드 리뷰, 에러 리뷰, 팀원 중도 이탈 상황에서의 업무 재분배 및 일정 재조정

깨달은 점

- DTO와 Entity를 분리하는 이유를 체감했다. 처음에는 Entity 하나로 요청·응답·영속성 까지 모두 처리해도 될 것 같았지만, 회원가입 시 노출되면 안 되는 필드·수정 시에만 오는 필드·응답에만 필요한 필드가 뒤섞이면서 금방 한계가 드러났다. 요청별 DTO를 분리한 뒤에야 Controller 코드가 단순해지고, 엔티티가 DB 관점에 집중할 수 있게 된다는 걸 실감했다.
- Interceptor는 "공통 관심사를 요청 처리 파이프라인에서 빼내는 도구"라는 걸 이해했다. 처음에는 각 Controller 메서드마다 세션 체크 코드를 `if` `(session.getAttribute(...) == null)` 식으로 반복해서 넣었다가, `LoginCheckInterceptor`

로 추출하고 `WebConfig` 에서 `excludePathPatterns` 로 예외 경로만 지정하는 방식으로 바꾸니 컨트롤러가 훨씬 깔끔해졌다.

- PM 역할은 기술 이상으로 "예외 상황 대응"이 핵심이라는 걸 배웠다. 5명으로 시작한 팀에서 한 명이 중도 이탈하면서 해당 기능의 분량을 나머지 인원이 나눠 맡아야 했다. 그때 가장 중요했던 건 남은 일정 대비 필수 기능과 포기 가능한 기능을 빠르게 재조정하는 일이었다. 완벽한 계획을 짜는 것보다, 변수가 생겼을 때 얼마나 빠르게 현실적인 재계획을 내놓을 수 있느냐가 PM의 실제 역량임을 처음으로 체감했다.